IV Advanced Models

From MLP to CNN and RNN

24

Traditional neural networks are the multilayer perceptron architecture. It is built with fully-connected layers. This type of layers is simple in algorithm but with a lot of parameters. In more advanced application of deep learning, you will see other kind of architecture. In this chapter, you will be introduced to two popular architectures of deep learning. After you finished this chapter, you will learn

- ▷ What is convolutional neural network (CNN)
- ▷ What is recurrent neural network (RNN)
- ▷ Why CNN and RNN can do better in some applications

Let's get started.

Overview

This chapter is in two parts; they are:

- ▷ Introduction to Convolutional Neural Network
- ▷ Introduction to Recurrent Neural Network

24.1 Introduction to Convolutional Neural Network

Convolutional neural networks (CNNs) are deep neural networks that are designed to work with image data. They are commonly used for object recognition, image classification, and image segmentation. The key building blocks of a CNN are convolutional layers and pooling layers. Usually fully connected layers are also used as output layers in CNN models.

Convolutional layers are responsible for feature extraction by applying a set of learnable filters to the input image. Unlike the fully-connected layers, convolutional layers produce output with a sliding window which the filter is applied repeatedly. Filters are multiplied with different regions of the input image to identify the presence of specific features. Because filters are applied in a sliding window, the features are *spatially invariant* meaning a particular element in the image (e.g., a cat) is not required to appear at a specific location. Also, because of the filters are applied in a window, they are usually much smaller than the image.

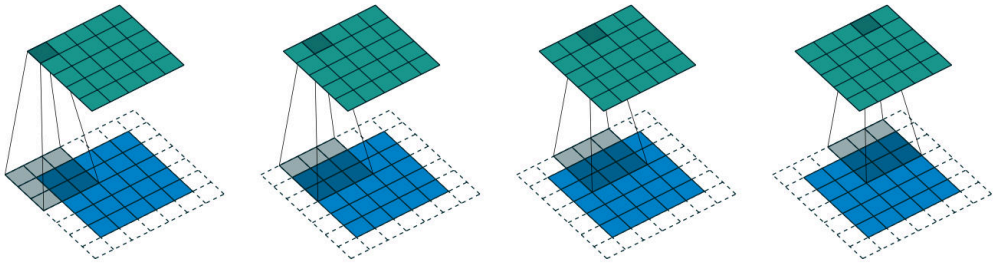


Figure 24.1: Operation of a convolutional layer: Blue is the input, gray is the filter, and green is the output. Image from "A guide to convolution arithmetic for deep learning"

Pooling layers in CNNs are to reduce the dimensionality of the feature maps generated by the convolutional layers. Similar to convolutional layers, it is applied with a sliding window. But instead of multiplying a filter, usually some aggregation function is used, such as taking the average or finding the maximum in the window. By carefully designing how the window slides, you can convert a high resolution image to lower resolution using pooling layer. This is how eventually you can reduce a million-pixel image into a binary classification (e.g., whether a cat is in the image), for example.

Fully-connected layers are used for classification by taking the output of the convolutional and pooling layers and passing it through a set of fully-connected neurons. Therefore, you usually see them near the output of the network model.

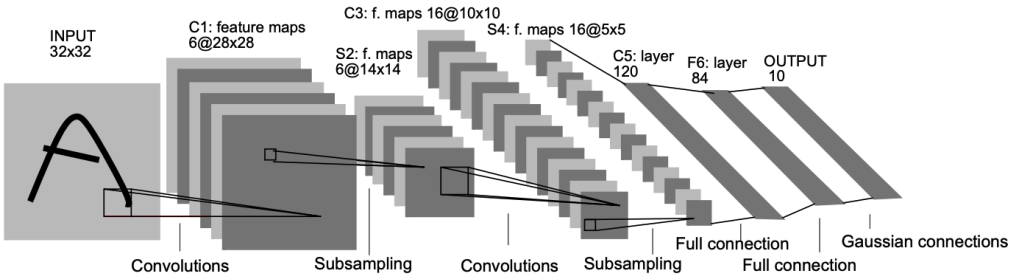


Figure 24.2: One of the earliest CNN, with convolutional layers, pooling layers (subsampling), and fully-connected layers. Image from "Gradient-Based Learning Applied to Document Recognition"

CNNs have been applied to a variety of image-related tasks such as object detection, facial recognition, and image segmentation. For example, it can be used to identify pedestrians, cars, and traffic signals on a street view to make a self-driving car.

CNNs are better suited for image-related tasks than Multilayer Perceptrons (MLPs) due to their ability to capture spatial information. MLPs process the input data as a flat vector, which means that they cannot take into account the spatial relationships between the different parts of an image. In contrast, CNNs keep the input image in two-dimensional and use convolutional layers to extract features from different regions of an image, allowing them to capture spatial information effectively.

Additionally, CNNs have fewer parameters than MLPs, which makes them more computationally efficient. MLPs require a large number of parameters to learn complex

patterns in data, whereas CNNs can learn the same patterns using fewer parameters. This makes CNNs more suitable for tasks that involve processing large amounts of data, such as image classification and object detection.

24.2 Introduction to Recurrent Neural Network

Recurrent neural networks (RNNs) are used for processing sequential data such as natural language text and speech. They are designed to capture the temporal dependencies between the input data.

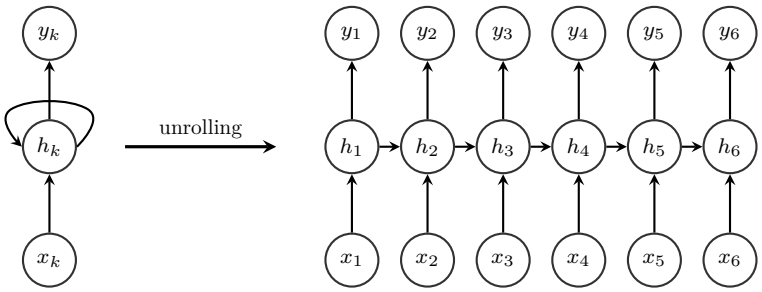


Figure 24.3: Structure of a recurrent neural network, before and after unrolling

The core building block of an RNN is the *recurrent layer*. The key idea behind a recurrent layer is to have one or more hidden state, which accumulates information about the input as the input sequence is provided. While the input is a sequence, the neural network is the same for each step, and produces output of the corresponding step. It is similar to an MLP processing one step of input at a time, except the hidden state is involved in producing the output and the input is indeed a sequence with many steps.

The hidden state of the recurrent layer is fed back into the layer at the next time step, allowing the network to maintain a memory of previous inputs. Therefore, sometimes you will see recurrent layer *unrolled* to emphasize the sequence structure of input and output. However, the repeating structure is indeed just one recurrent layer. The hidden state is not a learnable parameter, but rather, it is a state variable that is accumulated on the input sequence. It works as if it is the summary of the input sequence the RNN has seen so far. The processing of the input and the algorithm to update the hidden state is learned, but it is reused for every step of the input sequence. It is called *recurrent* because algorithmically, the recurrent layer is processing the input by applying the same algorithm in a loop on every step of the sequence.

RNNs can be more effective than Multilayer Perceptrons (MLPs) for sequential data tasks because they are designed to capture the temporal dependencies between the input data. MLPs process the input data as a flat vector, which means that they cannot take into account the order of the data. Moreover, since the input data is considered as a flat vector, inherently MLP is encouraged to process different time steps in the sequence differently. In contrast, RNNs use recurrent layers to maintain a memory of previous inputs, and consider each time step of the sequence as homogeneous, allowing them to capture the temporal dependencies in the data effectively.

The memory held by the hidden state makes RNNs well suited for tasks with variable length, such as language modeling, machine translation, and speech recognition. MLPs, on the other hand, require fixed-length inputs, which can be a limitation for tasks that involve variable-length data. In language processing, RNNs are used to generate, classify, and translate text. In speech recognition, RNN is used to convert spoken words into text, which RNN can analyze the sequence of sound waves produced by speech and identify the words being spoken.

24.3 Further Readings

This section provides more resources on the topic if you are looking to go deeper.

Vincent Dumoulin and Francesco Visin. "A guide to convolution arithmetic for deep learning", 2016.
https://arxiv.org/abs/1603.07285

Yann LeCun, Leon Bottou, Yoshua Bengio, and Patrick Haffner. "Gradient-Based Learning Applied to Document Recognition". In: *Proceedings of the IEEE*. Nov. 1998.
http://yann.lecun.com/exdb/publis/pdf/lecun-01a.pdf

Andrej Karpathy. *The Unreasonable Effectiveness of Recurrent Neural Networks*. May 2015.
http://karpathy.github.io/2015/05/21/rnn-effectiveness/

Recurrent neural network. Wikipedia.
https://en.wikipedia.org/wiki/Recurrent_neural_network

24.4 Summary

CNN and RNN are two powerful architectures of deep learning models that are widely used in computer vision and natural language processing respectively. Understanding these architectures is essential for building effective deep learning models for a wide range of applications.

In the next chapter, you will learn to implement a CNN using PyTorch.

Building a Convolutional Neural Network in PyTorch

25

Neural networks are built with layers connected to each other. There are different kinds of layers. For image related applications, you can always find convolutional layers. It is a layer with very few parameters but applied over a large sized input. It is powerful because it can preserve the spatial structure of the image. Therefore, it is used to produce state-of-the-art results on computer vision neural networks. In this chapter, you will learn about the convolutional layer and the network it built. After completing this chapter, you will know:

- ▷ What are convolutional layers and pooling layers
- ▷ How they fit together in a neural network
- ▷ How a neural network using convolutional layers is designed

Let's get started.

Overview

This chapter is divided into four parts; they are

- ▷ The Case for Convolutional Neural Networks
- ▷ Building Blocks of Convolutional Neural Networks
- ▷ An Example of Convolutional Neural Network
- ▷ What Are in Feature Maps?

25.1 The Case for Convolutional Neural Networks

Let's consider making a neural network to process grayscale image as input, which is the simplest use case in deep learning for computer vision.

A grayscale image is an array of pixels. Each pixel is usually a value in a range of 0 to 255. An image with size 32×32 would have 1024 pixels. Taking it as input to a neural network would mean the first layer to have at least 1024 input weights.

Looking at the pixel value has little use to understanding the picture, because data is hiding in the spatial structure (e.g., whether there is a horizontal line or a vertical line on

the picture). Hence the traditional neural network will find it difficult to find out information from image input.

Convolutional neural network is to use convolutional layers to preserve spatial information of pixels. It learns how alike are the neighboring pixels and generating *feature representations*. What the convolutional layers see from the picture is invariant to distortion in some degree. For example, a convolutional neural network could predict the same result even if the input image has shift in color, rotated or rescaled. Moreover, convolutional layers have fewer weights, thus easier to train.

25.2 Building Blocks of Convolutional Neural Networks

The simplest use case of a convolutional neural network is for classification. You will find it to contain three types of layers:

1. Convolutional layers
2. Pooling layers
3. Fully-connected layers

Neurons on a convolutional layer is called the filter. Usually it is a 2D convolutional layer in image application. The filter is a 2D patch (e.g., 3×3 pixels) that is applied on the input image pixels. The size of this 2D patch is also called the receptive field, meaning how large a portion of the image it can see at a time.

The filter of a convolutional layer is to multiply with the input pixels, and then sum up the result. This result is one pixel value at the output. The filter will move around the input image to fill out all pixel values at the output. Usually multiple filters are applied to the same input, producing multiple output tensors. These output tensors are called the *feature maps* produced by this layer. They are stacked together as one tensor and pass on to the next layer as input.

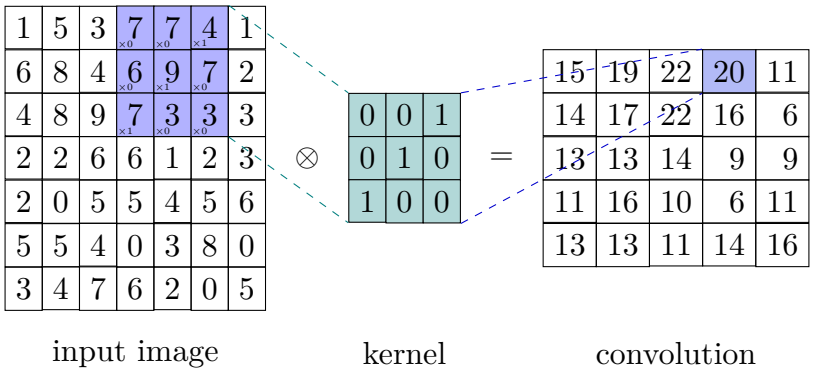


Figure 25.1: Example of a Filter Applied to a Two-Dimensional Input to Create a Feature Map

The output of a convolutional layer is called feature maps because usually it learned about the features of the input image. For example, whether there are vertical lines at the position. Learning the features from pixels is to help understanding the image at a higher

level. Multiple convolutional layers are stacked together in order to infer higher level features from lower level details.

Pooling layer is to *downsample* the previous layer's feature map. It is usually used after a convolutional layer to consolidate features learned. It can compress and generalize the feature representations. A pooling layer also has a receptive field and usually it is to take the average (average pooling) or the maximum (max pooling) over all values on the receptive field.

Fully connected layers are usually the final layers in a network. It is to take the features consolidated by previous convolutional and pooling layers as input to produce prediction. There might be multiple fully connected layers stacked together. In the case of classification, you usually see the output of the final fully connected layer applied with a softmax function to produce probability-like classification.

25.3 An Example of Convolutional Neural Network

The following is a program to do image classification on the CIFAR-10 dataset.

```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision

transform = torchvision.transforms.Compose([torchvision.transforms.ToTensor()])

trainset = torchvision.datasets.CIFAR10(root='./data', train=True,
                                        download=True, transform=transform)
testset = torchvision.datasets.CIFAR10(root='./data', train=False,
                                       download=True, transform=transform)

batch_size = 32
trainloader = torch.utils.data.DataLoader(trainset, batch_size=batch_size, shuffle=True)
testloader = torch.utils.data.DataLoader(testset, batch_size=batch_size, shuffle=True)

class CIFAR10Model(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 32, kernel_size=(3,3), stride=1, padding=1)
        self.act1 = nn.ReLU()
        self.drop1 = nn.Dropout(0.3)

        self.conv2 = nn.Conv2d(32, 32, kernel_size=(3,3), stride=1, padding=1)
        self.act2 = nn.ReLU()
        self.pool2 = nn.MaxPool2d(kernel_size=(2, 2))

        self.flat = nn.Flatten()

        self.fc3 = nn.Linear(8192, 512)
        self.act3 = nn.ReLU()
        self.drop3 = nn.Dropout(0.5)
```

```
        self.fc4 = nn.Linear(512, 10)

    def forward(self, x):
        # input 3x32x32, output 32x32x32
        x = self.act1(self.conv1(x))
        x = self.drop1(x)
        # input 32x32x32, output 32x32x32
        x = self.act2(self.conv2(x))
        # input 32x32x32, output 32x16x16
        x = self.pool2(x)
        # input 32x16x16, output 8192
        x = self.flat(x)
        # input 8192, output 512
        x = self.act3(self.fc3(x))
        x = self.drop3(x)
        # input 512, output 10
        x = self.fc4(x)
        return x

model = CIFAR10Model()
loss_fn = nn.CrossEntropyLoss()
optimizer = optim.SGD(model.parameters(), lr=0.001, momentum=0.9)

n_epochs = 20
for epoch in range(n_epochs):
    for inputs, labels in trainloader:
        # forward, backward, and then weight update
        y_pred = model(inputs)
        loss = loss_fn(y_pred, labels)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    acc = 0
    count = 0
    for inputs, labels in testloader:
        y_pred = model(inputs)
        acc += (torch.argmax(y_pred, 1) == labels).float().sum()
        count += len(labels)
    acc /= count
    print("Epoch %d: model accuracy %.2f%%" % (epoch, acc*100))

torch.save(model.state_dict(), "cifar10model.pth")
```

Listing 25.1: Classification model on CIFAR-10 image dataset

The CIFAR-10 dataset provides images in 32×32 pixels in RGB color (i.e., 3 color channels). There are 10 classes, labeled in integers 0 to 9. Whenever you are working on PyTorch neural network models for images, you will find the sister library `torchvision` useful. In the above, you used it to download the CIFAR-10 dataset from the Internet and transform it into a PyTorch tensor:

```
...
transform = torchvision.transforms.Compose([torchvision.transforms.ToTensor()])

trainset = torchvision.datasets.CIFAR10(root='./data', train=True,
                                        download=True, transform=transform)
testset = torchvision.datasets.CIFAR10(root='./data', train=False,
                                       download=True, transform=transform)
```

Listing 25.2: Load dataset and transform into PyTorch tensor

You also used a `DataLoader` in PyTorch to help creating batches for training. Training is to optimize the cross-entropy loss of the model, using stochastic gradient descent. It is a classification model, hence accuracy of classification is more intuitive than cross-entropy, which is computed at the end of each epoch, by comparing the maximum value in the output logit to the dataset's labels:

```
...
acc += (torch.argmax(y_pred, 1) == labels).float().sum()
```

Listing 25.3: Computing accuracy from logit output

It takes time to run the program above to train the network. This network should be able to achieve above 70% accuracy in classification.

It is typical in a network for image classification to be comprised of convolutional layers at an early stage, with dropout and pooling layers interleaved. Then, at a later stage, the output from convolutional layers is flattened and processed by some fully connected layers.

25.4 What Are in Feature Maps?

There are two convolutional layers in the network defined above. They are both defined with kernel size of 3×3, hence it is looking at 9 pixels at a time to produce one output pixel. Note that the first convolutional layer is taking the RGB image as input. Hence each pixel has three channels. The second convolutional layer is taking a feature map with 32 channels as input. Each "pixel" as it sees will have 32 values. Thus the second convolutional layer has more parameters even they have the same receptive field.

Let's see what is in the feature map. Let's say we pick one input sample from the training set:

```
import matplotlib.pyplot as plt

plt.imshow(trainset.data[7])
plt.show()
```

Listing 25.4: Showing one image from the dataset

You should see that this is an image of a horse, in 32×32 pixels with RGB channels:

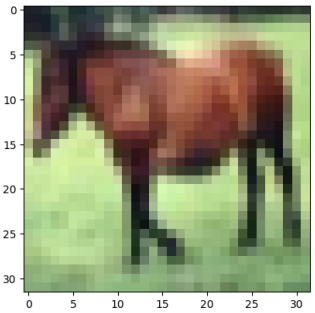


Figure 25.2: An example image

First, you need to convert this into a PyTorch tensor and make it a batch of one image. PyTorch models expect each image as a tensor in the format of (channel, height, width) but the data you read is in the format of (height, width, channel). If you use `torchvision` to transform the image into PyTorch tensors, this format conversion is done automatically. Otherwise, you need to *permute* the dimensions before use.

Afterward, pass it on through the model's first convolution layer and capture the output. You need to tell PyTorch that no gradient is needed for this calculation as you are not going to optimize the model weight:

```
X = torch.tensor([trainset.data[7]], dtype=torch.float32).permute(0,3,1,2)
model.eval()
with torch.no_grad():
    feature_maps = model.conv1(X)
```

Listing 25.5: Extracting feature map from a convolutional layer

The feature maps are in one tensor. You can visualize them using matplotlib:

```
fig, ax = plt.subplots(4, 8, sharex=True, sharey=True, figsize=(16,8))
for i in range(0, 32):
    row, col = i//8, i%8
    ax[row][col].imshow(feature_maps[0][i])
plt.show()
```

Listing 25.6: Visualizing the feature map

You may see the following:

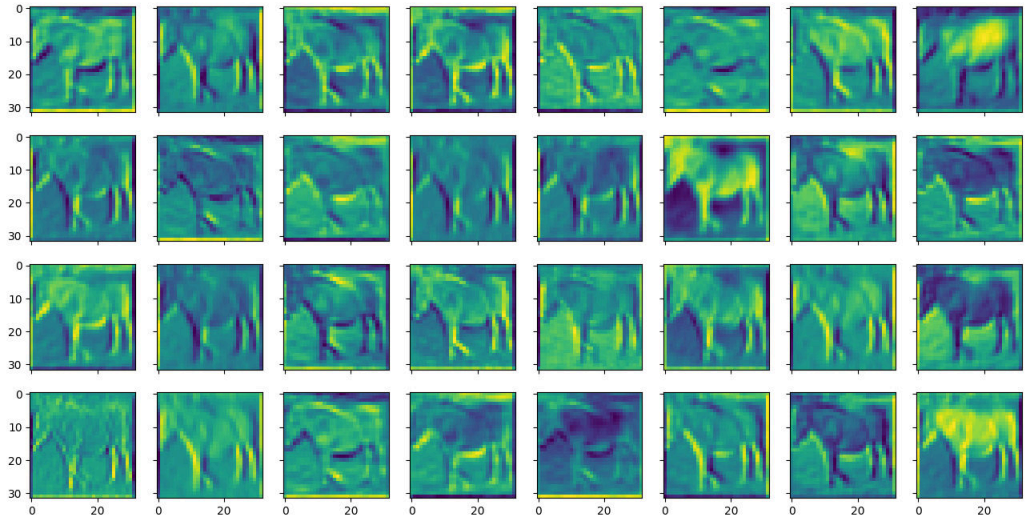


Figure 25.3: Feature map from an image

You can see that they are called feature maps because they are highlighting certain features from the input image. A feature is identified using a small window (in this case, over a 3×3 pixels filter). The input image has three color channels. Each channel has a different filter applied, and their results are combined for an output feature.

You can similarly display the feature map from the output of the second convolutional layer as follows:

```
X = torch.tensor([trainset.data[7]], dtype=torch.float32).permute(0,3,1,2)

model.eval()
with torch.no_grad():
    feature_maps = model.act1(model.conv1(X))
    feature_maps = model.drop1(feature_maps)
    feature_maps = model.conv2(feature_maps)

fig, ax = plt.subplots(4, 8, sharex=True, sharey=True, figsize=(16,8))
for i in range(0, 32):
    row, col = i//8, i%8
    ax[row][col].imshow(feature_maps[0][i])
plt.show()
```

Listing 25.7: Extracting feature map from the second convolutional layer

Which shows:

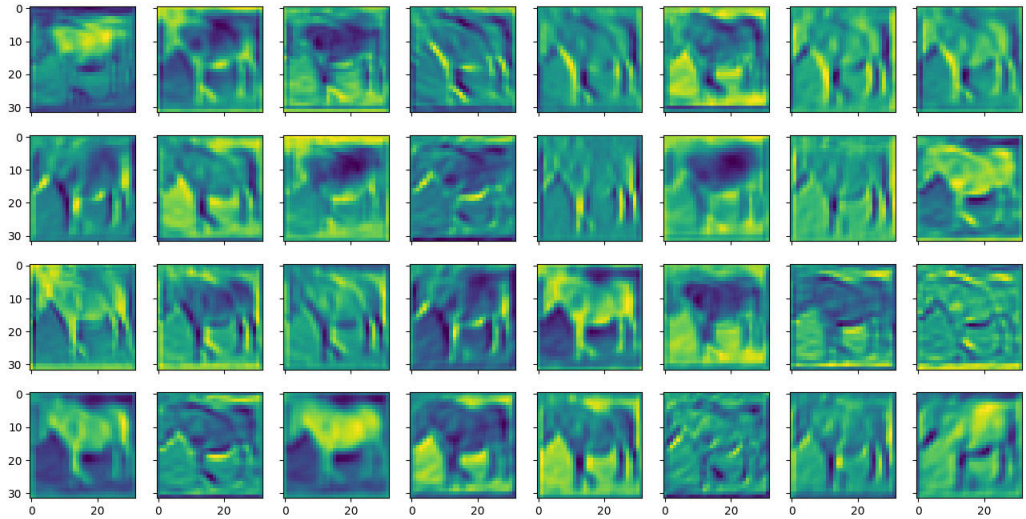


Figure 25.4: Feature map from the second convolutional layer

Compared to the output of the first convolutional layer, the feature maps from the second convolutional layer looks blurry and more abstract. But these are more useful for the model to identify the objects.

Putting everything together, the code below loads the saved model from the previous section and generate the feature maps:

```
import torch
import torch.nn as nn
import torchvision
import matplotlib.pyplot as plt

trainset = torchvision.datasets.CIFAR10(root='./data', train=True, download=True)

class CIFAR10Model(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 32, kernel_size=(3,3), stride=1, padding=1)
        self.act1 = nn.ReLU()
        self.drop1 = nn.Dropout(0.3)

        self.conv2 = nn.Conv2d(32, 32, kernel_size=(3,3), stride=1, padding=1)
        self.act2 = nn.ReLU()
        self.pool2 = nn.MaxPool2d(kernel_size=(2, 2))

        self.flat = nn.Flatten()

        self.fc3 = nn.Linear(8192, 512)
        self.act3 = nn.ReLU()
        self.drop3 = nn.Dropout(0.5)

        self.fc4 = nn.Linear(512, 10)

    def forward(self, x):
```

```
        # input 3x32x32, output 32x32x32
        x = self.act1(self.conv1(x))
        x = self.drop1(x)
        # input 32x32x32, output 32x32x32
        x = self.act2(self.conv2(x))
        # input 32x32x32, output 32x16x16
        x = self.pool2(x)
        # input 32x16x16, output 8192
        x = self.flat(x)
        # input 8192, output 512
        x = self.act3(self.fc3(x))
        x = self.drop3(x)
        # input 512, output 10
        x = self.fc4(x)
        return x

model = CIFAR10Model()
model.load_state_dict(torch.load("cifar10model.pth"))

plt.imshow(trainset.data[7])
plt.show()

X = torch.tensor([trainset.data[7]], dtype=torch.float32).permute(0,3,1,2)
model.eval()
with torch.no_grad():
    feature_maps = model.conv1(X)
fig, ax = plt.subplots(4, 8, sharex=True, sharey=True, figsize=(16,8))
for i in range(0, 32):
    row, col = i//8, i%8
    ax[row][col].imshow(feature_maps[0][i])
plt.show()

with torch.no_grad():
    feature_maps = model.act1(model.conv1(X))
    feature_maps = model.drop1(feature_maps)
    feature_maps = model.conv2(feature_maps)
fig, ax = plt.subplots(4, 8, sharex=True, sharey=True, figsize=(16,8))
for i in range(0, 32):
    row, col = i//8, i%8
    ax[row][col].imshow(feature_maps[0][i])
plt.show()
```

Listing 25.8: Load a saved model and extract feature maps

25.5 Further Readings

This section provides more resources on the topic if you are looking to go deeper.

Vincent Dumoulin and Francesco Visin. “A guide to convolution arithmetic for deep learning”, 2016.
https://arxiv.org/abs/1603.07285

Training a Classifier. PyTorch tutorials.
https://pytorch.org/tutorials/beginner/blitz/cifar10_tutorial.html
Conv2d. PyTorch API.
https://pytorch.org/docs/stable/generated/torch.nn.Conv2d.html
Ian Goodfellow, Yoshua Bengio, and Aaron Courville. Chapter 9, "Convolutional Networks". In: *Deep Learning*. MIT Press, 2016.
https://www.amazon.com/dp/0262035618
(Online version at http://www.deeplearningbook.org).

25.6 Summary

In this chapter, you learned how to use convolutional neural network to handle image input and how to visualize the feature maps.

Specifically, you learned:

- ▷ The structure of a typical convolutional neural networks
- ▷ What is the effect of the filter size to a convolutional layer
- ▷ What is the effect of stacking convolutional layers in a network
- ▷ How to extract and visualize feature maps from a convolutional neural network

In the next chapter, you will learn the classic example of using CNN to recognize handwritten digits.

Handwritten Digit Recognition with LeNet5 Model in PyTorch

26

A popular demonstration of the capability of deep learning techniques is object recognition in image data. The "hello world" of object recognition for machine learning and deep learning is the MNIST dataset for handwritten digit recognition. In this chapter, you will discover how to develop a deep learning model to achieve near state-of-the-art performance on the MNIST handwritten digit recognition task in PyTorch. After completing this chapter, you will know:

- ▷ How to load the MNIST dataset using torchvision
- ▷ How to develop and evaluate a baseline neural network model for the MNIST problem
- ▷ How to implement and evaluate a simple Convolutional Neural Network for MNIST
- ▷ How to implement a state-of-the-art deep learning model for MNIST

Let's get started.

Overview

This chapter is divided into five parts; they are:

- ▷ The MNIST Handwritten Digit Recognition Problem
- ▷ Loading the MNIST Dataset in PyTorch
- ▷ Baseline Model with Multilayer Perceptrons
- ▷ Simple Convolutional Neural Network for MNIST
- ▷ LeNet5 for MNIST

26.1 The MNIST Handwritten Digit Recognition Problem

The MNIST problem is a classic problem that can demonstrate the power of convolutional neural networks. The MNIST dataset was developed by Yann LeCun, Corinna Cortes, and Christopher Burges for evaluating machine learning models on the handwritten digit classification problem. The dataset was constructed from a number of scanned document datasets available from the National Institute of Standards and Technology (NIST). This is where the name for the dataset comes from, the Modified NIST or MNIST dataset.

Images of digits were taken from a variety of scanned documents, normalized in size, and centered. This makes it an excellent dataset for evaluating models, allowing the developer to focus on machine learning with minimal data cleaning or preparation required. Each image is a 28×28-pixel square (784 pixels total) in grayscale. A standard split of the dataset is used to evaluate and compare models, where 60,000 images are used to train a model, and a separate set of 10,000 images are used to test it.

To goal of this problem is to identify the digits on the image. There are ten digits (0 to 9) or ten classes to predict. The state-of-the-art prediction accuracy is at 99.8% level, achieved with large convolutional neural networks.

26.2 Loading the MNIST Dataset in PyTorch

The `torchvision` library is a sister project of PyTorch that provide specialized functions for computer vision tasks. There is a function in `torchvision` that can download the MNIST dataset for use with PyTorch. The dataset is downloaded the first time this function is called and stored locally, so you don't need to download again in the future. Below is a little script to download and visualize the first 16 images in the training subset of the MNIST dataset.

```
import matplotlib.pyplot as plt
import torchvision

train = torchvision.datasets.MNIST('./data', train=True, download=True)

fig, ax = plt.subplots(4, 4, sharex=True, sharey=True)
for i in range(4):
    for j in range(4):
        ax[i][j].imshow(train.data[4*i+j], cmap="gray")
plt.show()
```

Listing 26.1: Loading the MNIST dataset and display some samples

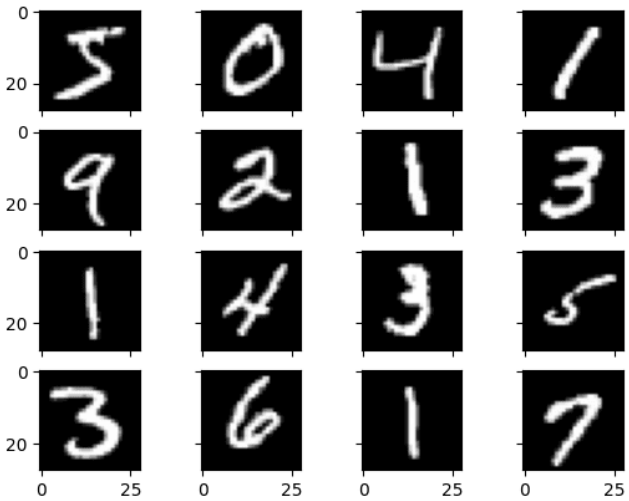


Figure 26.1: Examples from the MNIST dataset

26.3 Baseline Model with Multilayer Perceptrons

Do you really need a complex model like a convolutional neural network to get the best results with MNIST? You can get good results using a very simple neural network model with a single hidden layer. In this section, you will create a simple multilayer perceptron model that achieves accuracy of 99.81%. You will use this as a baseline for comparison to more complex convolutional neural network models. First, let's check what the data looks like:

```
import torchvision

# Load MNIST data
train = torchvision.datasets.MNIST('data', train=True, download=True)
test = torchvision.datasets.MNIST('data', train=False, download=True)
print(train.data.shape, train.targets.shape)
print(test.data.shape, test.targets.shape)
```

Listing 26.2: Showing the shape of the MNIST dataset

You should see:

```
torch.Size([60000, 28, 28]) torch.Size([60000])
torch.Size([10000, 28, 28]) torch.Size([10000])
```

Output 26.1: Shape of the MNIST dataset

The training dataset is structured as a 3-dimensional array of instance, image height, and image width. For a multilayer perceptron model, you must reduce the images down into a vector of pixels. In this case, the 28 × 28-sized images will be 784 pixel input vectors. You can do this transform easily using the `reshape()` function.

The pixel values are grayscale between 0 and 255. It is almost always a good idea to perform some scaling of input values when using neural network models. Because the scale is well known and well behaved, you can very quickly normalize the pixel values to the range 0 and 1 by dividing each value by the maximum of 255.

In the following, you transform the dataset, convert to floating point, and normalize them by scaling floating point values and you can normalize them easily in the next step.

```
# each sample becomes a vector of values 0-1
X_train = train.data.reshape(-1, 784).float() / 255.0
y_train = train.targets
X_test = test.data.reshape(-1, 784).float() / 255.0
y_test = test.targets
```

Listing 26.3: Rescaling the dataset

The output targets `y_train` and `y_test` are labels in the form of integers from 0 to 9. This is a multiclass classification problem. You can convert these labels into one-hot encoding or keep them as integer labels like this case. You are going to use the cross-entropy function to evaluate the model performance and the PyTorch implementation of cross-entropy function can be applied on one-hot encoded targets or integer labeled targets.

You are now ready to create your simple neural network model. You will define your model in a PyTorch `Module` class.

```
class Baseline(nn.Module):
    def __init__(self):
        super().__init__()
        self.layer1 = nn.Linear(784, 784)
        self.act1 = nn.ReLU()
        self.layer2 = nn.Linear(784, 10)

    def forward(self, x):
        x = self.act1(self.layer1(x))
        x = self.layer2(x)
        return x
```

Listing 26.4: Baseline MLP model for classification of the MNIST dataset

The model is a simple neural network with one hidden layer with the same number of neurons as there are inputs (784). A rectifier activation function is used for the neurons in the hidden layer. The output of this model are *logits*, meaning they are real numbers which can be transformed into probability-like values using a softmax function. You do not apply the softmax function explicitly because the cross-entropy function will do that for you.

You will use the stochastic gradient descent algorithm (with learning rate set to 0.01) to optimize this model. The training loop is as follows:

```
model = Baseline()

optimizer = optim.SGD(model.parameters(), lr=0.01)
loss_fn = nn.CrossEntropyLoss()
loader = torch.utils.data.DataLoader(list(zip(X_train, y_train)),
                                     shuffle=True, batch_size=100)

n_epochs = 10
for epoch in range(n_epochs):
    model.train()
    for X_batch, y_batch in loader:
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
    # Validation
    model.eval()
    y_pred = model(X_test)
    acc = (torch.argmax(y_pred, 1) == y_test).float().mean()
    print("Epoch %d: model accuracy %.2f%%" % (epoch, acc*100))
```

Listing 26.5: Training the baseline model with SGD

The MNIST dataset is small. This example should complete in a minute, with the output below. This simple network can produce accuracy at 92%.

```
Epoch 0: model accuracy 84.11%
Epoch 1: model accuracy 87.53%
Epoch 2: model accuracy 89.01%
Epoch 3: model accuracy 89.76%
Epoch 4: model accuracy 90.29%
Epoch 5: model accuracy 90.69%
Epoch 6: model accuracy 91.10%
Epoch 7: model accuracy 91.48%
Epoch 8: model accuracy 91.74%
Epoch 9: model accuracy 91.96%
```

Output 26.2: Progress of the model training

Below is the complete code for the above multilayer perceptron classification on MNIST dataset.

```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision

# Load MNIST data
train = torchvision.datasets.MNIST('data', train=True, download=True)
test = torchvision.datasets.MNIST('data', train=False, download=True)

# each sample becomes a vector of values 0-1
X_train = train.data.reshape(-1, 784).float() / 255.0
y_train = train.targets
X_test = test.data.reshape(-1, 784).float() / 255.0
y_test = test.targets

class Baseline(nn.Module):
    def __init__(self):
        super().__init__()
        self.layer1 = nn.Linear(784, 784)
        self.act1 = nn.ReLU()
        self.layer2 = nn.Linear(784, 10)

    def forward(self, x):
        x = self.act1(self.layer1(x))
        x = self.layer2(x)
        return x

model = Baseline()

optimizer = optim.SGD(model.parameters(), lr=0.01)
loss_fn = nn.CrossEntropyLoss()
loader = torch.utils.data.DataLoader(list(zip(X_train, y_train)),
                                     shuffle=True, batch_size=100)

n_epochs = 10
for epoch in range(n_epochs):
    model.train()
    for X_batch, y_batch in loader:
```

```
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
    # Validation
    model.eval()
    y_pred = model(X_test)
    acc = (torch.argmax(y_pred, 1) == y_test).float().mean()
    print("Epoch %d: model accuracy %.2f%%" % (epoch, acc*100))
```

Listing 26.6: Complete code of building MNIST classification baseline model

26.4 Simple Convolutional Neural Network for MNIST

Now that you have seen how to use multilayer perceptron model to classify MNIST dataset. Let's move on to try a convolutional neural network model. In this section, you will create a simple CNN for MNIST that demonstrates how to use all the aspects of a modern CNN implementation, including convolutional layers, pooling layers, and dropout layers.

In PyTorch, convolutional layers are supposed to work on images. Tensors for images should be the pixel values with the dimensions (sample, channel, height, width) but when you load images using libraries such as PIL, the pixels are usually presented as array of dimensions (height, width, channel). The conversion to a proper tensor format can be done using a transform from the `torchvision` library.

```
...
transform = torchvision.transforms.Compose([
    torchvision.transforms.ToTensor(),
    torchvision.transforms.Normalize((0,), (128,)),
])
train = torchvision.datasets.MNIST('data', train=True, download=True, transform=transform)
test = torchvision.datasets.MNIST('data', train=False, download=True, transform=transform)
trainloader = torch.utils.data.DataLoader(train, shuffle=True, batch_size=100)
testloader = torch.utils.data.DataLoader(test, shuffle=True, batch_size=100)
```

Listing 26.7: Creating a `DataLoader` for MNIST dataset

You need to use `DataLoader` because the transform is applied when you read the data from the `DataLoader`.

Next, define your neural network model. Convolutional neural networks are more complex than standard multilayer perceptrons, so you will start by using a simple structure that uses all the elements for state-of-the-art results. Below summarizes the network architecture.

1. The first hidden layer is a convolutional layer, `nn.Conv2d()`. The layer turns a grayscale image into 10 feature maps, with the filter size of 5×5 and a ReLU activation function. This is the input layer that expects images with the structure outlined above.
2. Next is a pooling layer that takes the max, `nn.MaxPool2d()`. It is configured with a pool size of 2×2 with stride 1. What it does is to take the maximum in a 2×2

pixel patch per channel and assign the value to the output pixel. The result is a 27×27-pixels feature map per channel.

3. The next layer is a regularization layer using dropout, `nn.Dropout()`. It is configured to randomly exclude 20% of neurons in the layer in order to reduce overfitting.
4. Next is a layer that converts the 2D matrix data to a vector, using `nn.Flatten`. There are 10 channels from its input and each channel's feature map has size 27×27. This layer allows the output to be processed by standard, fully connected layers.
5. Next is a fully connected layer with 128 neurons. ReLU activation function is used.
6. Finally, the output layer has ten neurons for the ten classes. You can transform the output into probability-like predictions by applying a softmax function on it.

This model is trained using cross-entropy loss and the Adam optimiztion algorithm. It is implemented as follows:

```
class CNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv = nn.Conv2d(1, 10, kernel_size=5, stride=1, padding=2)
        self.relu1 = nn.ReLU()
        self.pool = nn.MaxPool2d(kernel_size=2, stride=1)
        self.dropout = nn.Dropout(0.2)
        self.flat = nn.Flatten()
        self.fc = nn.Linear(27*27*10, 128)
        self.relu2 = nn.ReLU()
        self.output = nn.Linear(128, 10)

    def forward(self, x):
        x = self.relu1(self.conv(x))
        x = self.pool(x)
        x = self.dropout(x)
        x = self.relu2(self.fc(self.flat(x)))
        x = self.output(x)
        return x

model = CNN()

optimizer = optim.Adam(model.parameters(), lr=0.01)
loss_fn = nn.CrossEntropyLoss()

n_epochs = 10
for epoch in range(n_epochs):
    model.train()
    for X_batch, y_batch in trainloader:
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
    # Validation
    model.eval()
```

```
    acc = 0
    count = 0
    for X_batch, y_batch in testloader:
        y_pred = model(X_batch)
        acc += (torch.argmax(y_pred, 1) == y_batch).float().sum()
        count += len(y_batch)
    acc = acc / count
    print("Epoch %d: model accuracy %.2f%%" % (epoch, acc*100))
```

Listing 26.8: Training a convolutional neural network model for MNIST

Running the above takes a few minutes and produces the following:

```
Epoch 0: model accuracy 81.74%
Epoch 1: model accuracy 85.38%
Epoch 2: model accuracy 86.37%
Epoch 3: model accuracy 87.75%
Epoch 4: model accuracy 88.00%
Epoch 5: model accuracy 88.17%
Epoch 6: model accuracy 88.81%
Epoch 7: model accuracy 88.34%
Epoch 8: model accuracy 88.86%
Epoch 9: model accuracy 88.75%
```

Output 26.3: Training progress of the CNN

Not the best result but this demonstrates how convolutional layer works. Below is the complete code for using the simple convolutional network.

```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision

# Load MNIST data
transform = torchvision.transforms.Compose([
    torchvision.transforms.ToTensor(),
    torchvision.transforms.Normalize((0,), (128,)),
])
train = torchvision.datasets.MNIST('data', train=True, download=True, transform=transform)
test = torchvision.datasets.MNIST('data', train=False, download=True, transform=transform)
trainloader = torch.utils.data.DataLoader(train, shuffle=True, batch_size=100)
testloader = torch.utils.data.DataLoader(test, shuffle=True, batch_size=100)

class CNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv = nn.Conv2d(1, 10, kernel_size=5, stride=1, padding=2)
        self.relu1 = nn.ReLU()
        self.pool = nn.MaxPool2d(kernel_size=2, stride=1)
        self.dropout = nn.Dropout(0.2)
        self.flat = nn.Flatten()
        self.fc = nn.Linear(27*27*10, 128)
        self.relu2 = nn.ReLU()
```

```
        self.output = nn.Linear(128, 10)

    def forward(self, x):
        x = self.relu1(self.conv(x))
        x = self.pool(x)
        x = self.dropout(x)
        x = self.relu2(self.fc(self.flat(x)))
        x = self.output(x)
        return x

model = CNN()

optimizer = optim.Adam(model.parameters())
loss_fn = nn.CrossEntropyLoss()

n_epochs = 10
for epoch in range(n_epochs):
    model.train()
    for X_batch, y_batch in trainloader:
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
    # Validation
    model.eval()
    acc = 0
    count = 0
    for X_batch, y_batch in testloader:
        y_pred = model(X_batch)
        acc += (torch.argmax(y_pred, 1) == y_batch).float().sum()
        count += len(y_batch)
    acc = acc / count
    print("Epoch %d: model accuracy %.2f%%" % (epoch, acc*100))
```

Listing 26.9: Complete code for classification using CNN

26.5 LeNet5 for MNIST

The previous model has only one convolutional layer. Of course, you can add more to make a deeper model. One of the earliest demonstration of the effectiveness of convolutional layers in neural networks is the "LeNet5" model. This model is developed to solve the MNIST classification problem. It has three convolutional layers and two fully connected layer to make up five trainable layers in the model, as it is named.

At the time it was developed, using hyperbolic tangent function as activation is common. Hence it is used here. This model is implemented as follows:

```
class LeNet5(nn.Module):
    def __init__(self):
        super().__init__()
```

```
        self.conv1 = nn.Conv2d(1, 6, kernel_size=5, stride=1, padding=2)
        self.act1 = nn.Tanh()
        self.pool1 = nn.AvgPool2d(kernel_size=2, stride=2)

        self.conv2 = nn.Conv2d(6, 16, kernel_size=5, stride=1, padding=0)
        self.act2 = nn.Tanh()
        self.pool2 = nn.AvgPool2d(kernel_size=2, stride=2)

        self.conv3 = nn.Conv2d(16, 120, kernel_size=5, stride=1, padding=0)
        self.act3 = nn.Tanh()

        self.flat = nn.Flatten()
        self.fc1 = nn.Linear(1*1*120, 84)
        self.act4 = nn.Tanh()
        self.fc2 = nn.Linear(84, 10)

    def forward(self, x):
        # input 1x28x28, output 6x28x28
        x = self.act1(self.conv1(x))
        # input 6x28x28, output 6x14x14
        x = self.pool1(x)
        # input 6x14x14, output 16x10x10
        x = self.act2(self.conv2(x))
        # input 16x10x10, output 16x5x5
        x = self.pool2(x)
        # input 16x5x5, output 120x1x1
        x = self.act3(self.conv3(x))
        # input 120x1x1, output 84
        x = self.act4(self.fc1(self.flat(x)))
        # input 84, output 10
        x = self.fc2(x)
        return x
```

Listing 26.10: LeNet5 model implemented in PyTorch

Compare to the previous model, LeNet5 does not have Dropout layer (because Dropout layer was invented several years after LeNet5) and use average pooling instead of max pooling (i.e., for a patch of 2×2 pixels, it is taking average of the pixel values instead of taking the maximum). But the most notable characteristic of LeNet5 model is that it uses strides and paddings to reduce the image size from 28×28 pixel down to 1×1 pixel while increasing the number of channels from a one (grayscale) into 120.

Padding means to add pixels of value 0 at the border of the image to make it a bit larger. Without padding, the output of a convolutional layer will be smaller than its input. The stride parameter controls how much the filter should move to produce the next pixel in the output. Usually it is 1 to preserve the same size. If it is larger than 1, the output is a *downsampling* of the input. Hence you see in the LeNet5 model, stride 2 was used in the pooling layers to make, for example, a 28×28-pixel image into 14×14.

Training this model is same as training the previous convolutional network model, as follows:

```
...
model = LeNet5()

optimizer = optim.Adam(model.parameters())
loss_fn = nn.CrossEntropyLoss()

n_epochs = 10
for epoch in range(n_epochs):
    model.train()
    for X_batch, y_batch in trainloader:
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
    # Validation
    model.eval()
    acc = 0
    count = 0
    for X_batch, y_batch in testloader:
        y_pred = model(X_batch)
        acc += (torch.argmax(y_pred, 1) == y_batch).float().sum()
        count += len(y_batch)
    acc = acc / count
    print("Epoch %d: model accuracy %.2f%%" % (epoch, acc*100))
```

Listing 26.11: Training the LeNet5 model

Running this you may see:

```
Epoch 0: model accuracy 89.46%
Epoch 1: model accuracy 93.14%
Epoch 2: model accuracy 94.69%
Epoch 3: model accuracy 95.84%
Epoch 4: model accuracy 96.43%
Epoch 5: model accuracy 96.99%
Epoch 6: model accuracy 97.14%
Epoch 7: model accuracy 97.66%
Epoch 8: model accuracy 98.05%
Epoch 9: model accuracy 98.22%
```

Output 26.4: Progress of training the LeNet5 model

Here, we achieved accuracy beyond 98%.

The following is the complete code.

```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision

# Load MNIST data
transform = torchvision.transforms.Compose([
```

```
    torchvision.transforms.ToTensor(),
    torchvision.transforms.Normalize((0,), (128,)),
])
train = torchvision.datasets.MNIST('data', train=True, download=True, transform=transform)
test = torchvision.datasets.MNIST('data', train=False, download=True, transform=transform)
trainloader = torch.utils.data.DataLoader(train, shuffle=True, batch_size=100)
testloader = torch.utils.data.DataLoader(test, shuffle=True, batch_size=100)

class LeNet5(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 6, kernel_size=5, stride=1, padding=2)
        self.act1 = nn.Tanh()
        self.pool1 = nn.AvgPool2d(kernel_size=2, stride=2)

        self.conv2 = nn.Conv2d(6, 16, kernel_size=5, stride=1, padding=0)
        self.act2 = nn.Tanh()
        self.pool2 = nn.AvgPool2d(kernel_size=2, stride=2)

        self.conv3 = nn.Conv2d(16, 120, kernel_size=5, stride=1, padding=0)
        self.act3 = nn.Tanh()

        self.flat = nn.Flatten()
        self.fc1 = nn.Linear(1*1*120, 84)
        self.act4 = nn.Tanh()
        self.fc2 = nn.Linear(84, 10)

    def forward(self, x):
        # input 1x28x28, output 6x28x28
        x = self.act1(self.conv1(x))
        # input 6x28x28, output 6x14x14
        x = self.pool1(x)
        # input 6x14x14, output 16x10x10
        x = self.act2(self.conv2(x))
        # input 16x10x10, output 16x5x5
        x = self.pool2(x)
        # input 16x5x5, output 120x1x1
        x = self.act3(self.conv3(x))
        # input 120x1x1, output 84
        x = self.act4(self.fc1(self.flat(x)))
        # input 84, output 10
        x = self.fc2(x)
        return x

model = LeNet5()

optimizer = optim.Adam(model.parameters())
loss_fn = nn.CrossEntropyLoss()

n_epochs = 10
for epoch in range(n_epochs):
    model.train()
    for X_batch, y_batch in trainloader:
        y_pred = model(X_batch)
```

```
        loss = loss_fn(y_pred, y_batch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
    # Validation
    model.eval()
    acc = 0
    count = 0
    for X_batch, y_batch in testloader:
        y_pred = model(X_batch)
        acc += (torch.argmax(y_pred, 1) == y_batch).float().sum()
        count += len(y_batch)
    acc = acc / count
    print("Epoch %d: model accuracy %.2f%%" % (epoch, acc*100))
```

Listing 26.12: Complete code of classification using LeNet5 model

26.6 Resources on MNIST

The MNIST dataset is very well studied. Below are some additional resources you might want to look into.

Yann LeCun, Corinna Cortes, and Christopher J. C. Burges. *The MNIST database of handwritten digits.*
http://yann.lecun.com/exdb/mnist/

Rodrigo Benenson. *What is the class of this image?* Classification datasets results, 2016.
https://rodrigob.github.io/are_we_there_yet/build/classification_datasets_results.html

Digit Recognizer: Learn computer vision fundamentals with the famous MNIST data. Kaggle.
https://www.kaggle.com/c/digit-recognizer

Hubert Eichner. *Neural Net for Handwritten Digit Recognition in JavaScript.*
http://myselph.de/neuralNet.html

Yann LeCun, Leon Bottou, Yoshua Bengio, and Patrick Haffner. "Gradient-Based Learning Applied to Document Recognition". In: *Proceedings of the IEEE*. Nov. 1998.
http://yann.lecun.com/exdb/publis/pdf/lecun-01a.pdf

Yann LeCun. *LeNet-5 convolutional neural networks.*
http://yann.lecun.com/exdb/lenet/

26.7 Summary

In this chapter, you discovered the MNIST handwritten digit recognition problem and deep learning models developed in Python using the PyTorch library that are capable of achieving excellent results. Working through this chapter, you learned:

- How to load the MNIST dataset in PyTorch with torchvision
- How to convert the MNIST dataset into PyTorch tensors for consumption by a convolutional neural network

- ▷ How to use PyTorch to create convolutional neural network models for MNIST
- ▷ How to implement the LeNet5 model for MNIST classification

In the next chapter, you will learn about LSTM, the most common type of recurrent neural network.